UNIT III

Trees and Balanced Trees: Representation of Binary Trees using Arrays and Linked Lists, Binary Tree Traversals (Inorder, Preorder, Postorder), Binary Search Trees, AVL Trees, 2-3 Trees, Heaps, Heap Sort

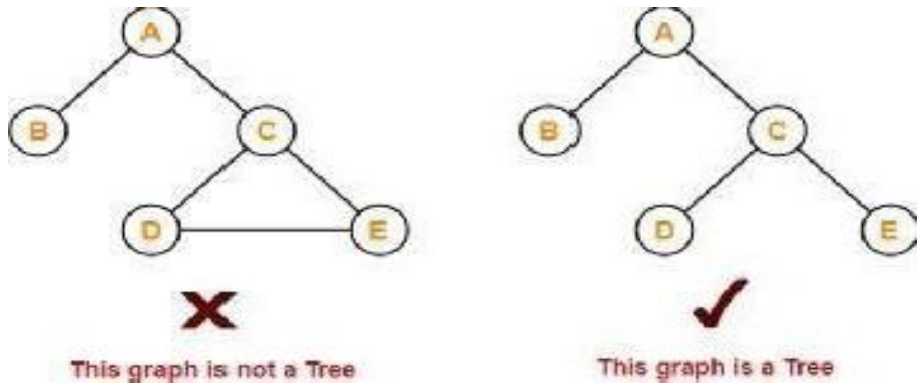
Introduction to trees:

TREE:

A tree is a non-linear data structure which organizes the data in hierarchical order.
It is an abstract model of hierarchical structure.
It consists of nodes with parent-child relation.
Each individual element is called node.
Each node stores the actual data and link to the next element in hierarchical structure.

Properties: The important properties of tree data structure are

There is one and only one path between every pair of vertices in a tree.
A tree with n vertices has exactly (n-1) edges.



Applications:

Tree is a very popular data structure used in wide range of applications. Some of them are:

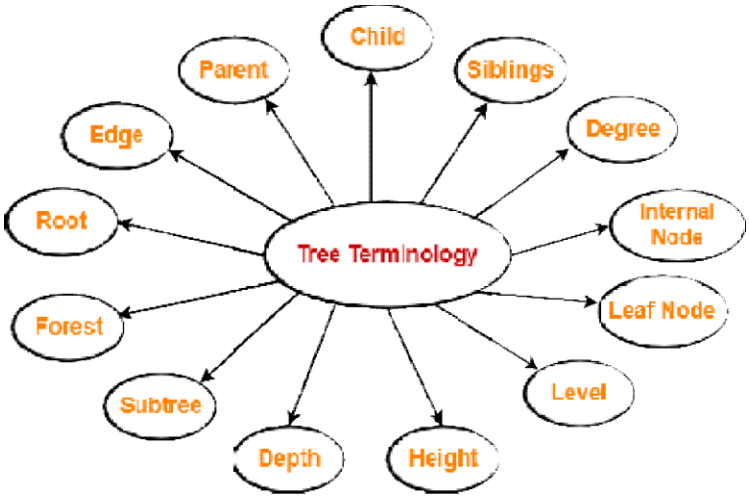
File system
Organization chat
Programming environments
Manipulate sorted lists of data
Easy to search by using tree traversals
Easy to search locations in maps.

Advantages:

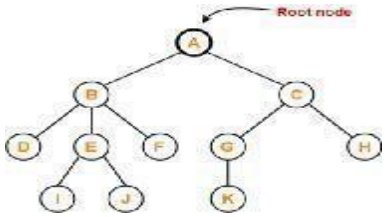
Accessing elements in trees is faster than linked lists.
Sorted binary trees gives best performance for searching.
By using trees, we can easily perform different operations on files and folders like creation, deletion etc.

1. Explain about Tree Terminologies

Following are important terms with respect to tree.

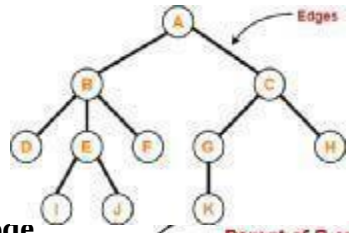


Root:



The first node from where the tree originates is called as a **root node**.
In any tree, there must be only one root node.
We can never have multiple root nodes in a tree data structure.

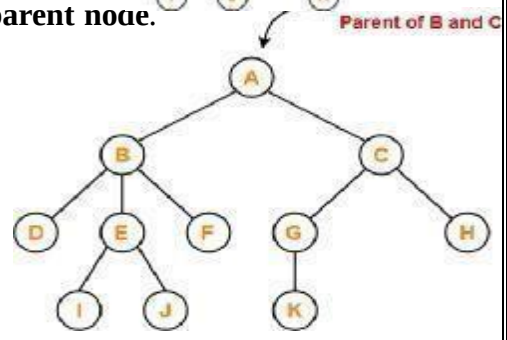
Edge:



The connecting link between any two nodes is called as an **edge**.
In a tree with n number of nodes, there is exactly (n-1) number of edges.

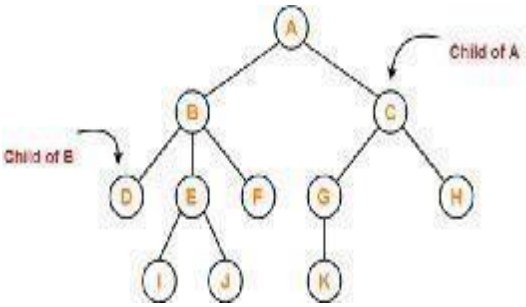
Parent:

The node which has a branch from it to any other node is called as a **parent node**.
In other words, the node which has one or more children is called as a parent node.
In a tree, a parent node can have any number of child nodes.
Here,
Node A is the parent of nodes B and C
Node B is the parent of nodes D, E and F
Node C is the parent of nodes G and H
Node E is the parent of nodes I and J
Node G is the parent of node K



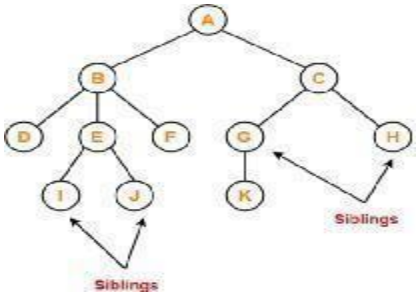
Child:

The node which is a descendant of some node is called as a **child node**.
All the nodes except root node are child nodes.
Here,
Nodes B and C are the children of node A
Nodes D, E and F are the children of node B
Nodes G and H are the children of node C
Nodes I and J are the children of node E
Node K is the child of node G



Siblings:

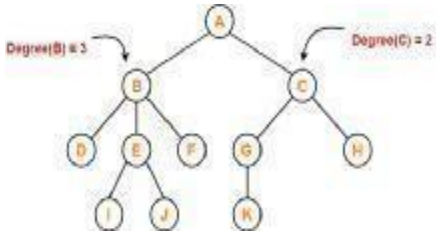
Nodes which belong to the same parent are called as **siblings**.
In other words, nodes with the same parent are sibling nodes.
Here,
Nodes B and C are siblings
Nodes D, E and F are siblings
Nodes G and H are siblings
Nodes I and J are siblings



Degree:

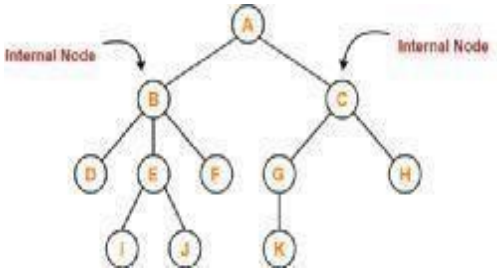
Degree of a node is the total number of children of that node.
Degree of a tree is the highest degree of a node among all the nodes in the tree.
Here,

Degree of node A = 2	Degree of node B = 3
Degree of node C = 2	Degree of node D = 0
Degree of node E = 2	Degree of node G = 1
Degree of node H = 0	Degree of node I = 0
Degree of node J = 0	Degree of node K = 0



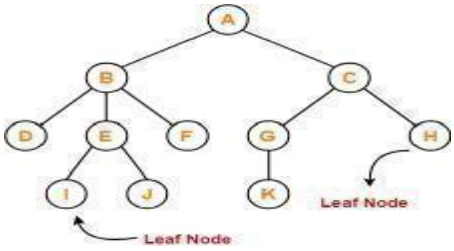
Internal Node:

The node which has at least one child is called as an **internal node**.
Internal nodes are also called as **non-terminal nodes**.
Every non-leaf node is an internal node.



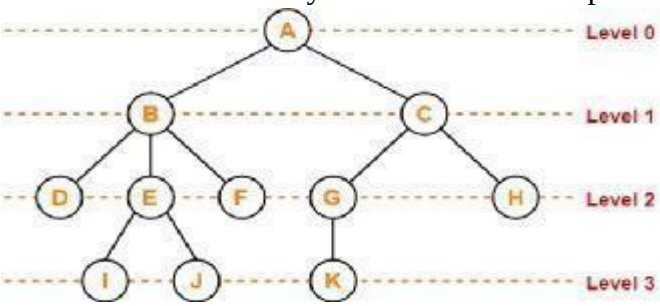
Leaf Node:

The node which does not have any child is called as a **leaf node**.
Leaf nodes are also called as **external nodes** or **terminal nodes**.



Level:

In a tree, each step from top to bottom is called as **level of a tree**.
The level count starts with 0 and increments by 1 at each level or step.

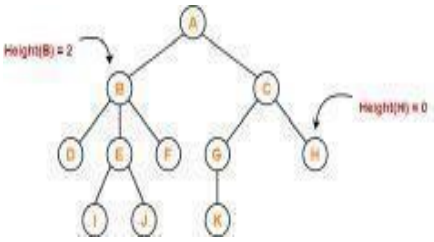


Height:

Total number of edges that lies on the longest path from any leaf node to a particular node is called as **height of that node**.
Height of a tree is the height of root node.
Height of all leaf nodes = 0

Here,

Height of node A = 3
Height of node B = 2
Height of node C = 2
Height of node D = 0
Height of node E = 1
Height of node F = 0
Height of node G = 1
Height of node H = 0
Height of node I = 0
Height of node J = 0
Height of node K = 0



Depth:

Total number of edges from root node to a particular node is called as **depth of that node**.

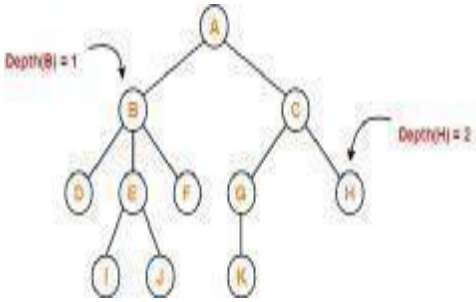
Depth of a tree is the total number of edges from root node to a leaf node in the longest path.

Depth of the root node = 0

The terms "level" and "depth" are used interchangeably.

Here,

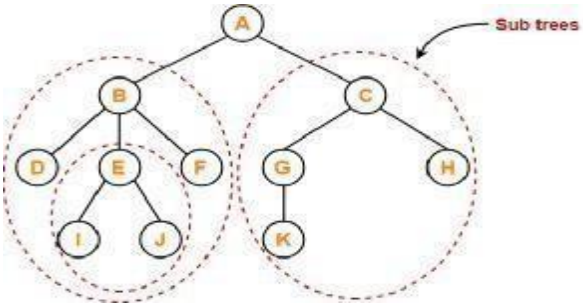
Depth of node A = 0
Depth of node B = 1
Depth of node C = 1
Depth of node D = 2
Depth of node E = 2
Depth of node F = 2
Depth of node G = 2
Depth of node H = 2
Depth of node I = 3
Depth of node J = 3
Depth of node K = 3



Subtree:

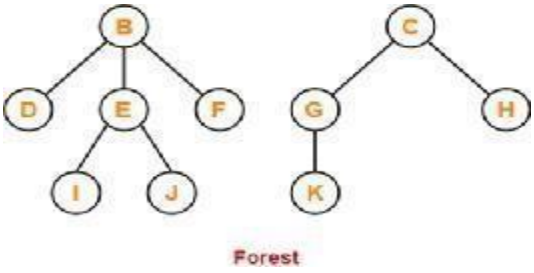
In a tree, each child from a node forms a **subtree** recursively.

Every child node forms a subtree on its parent node.



Forest:

A Forest is a disjoint of Trees



2. Explain about different types of trees?

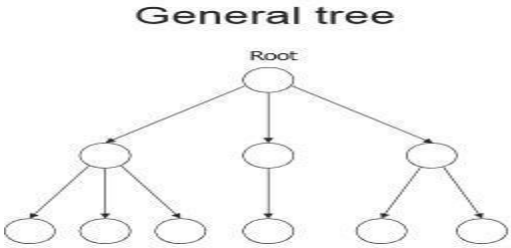
Tree: Tree is a non-linear DS, Which organizes the data in hierarchical form.

Types of trees: There are many different types of trees in DS. Some of them are:

1) General tree
2) Binary tree (BT)
3) Binary search tree (BST)

1. **General tree:**

A general tree is a tree where every node can have two or more children.



2. Binary tree:

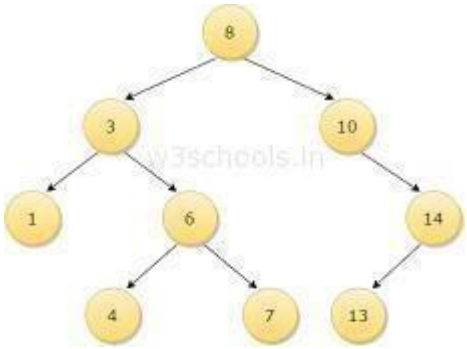
A binary tree is a tree where each node can have maximum of two children

It is defined as:

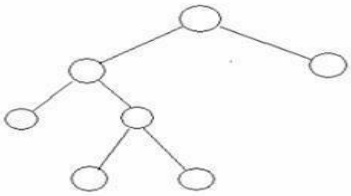
It is either empty (Or) It consists of a node called root node with two disjoint binary trees called left-sub tree and right-sub tree.

There are different types of binary trees available in DS. They are:

i. Strictly (OR) fully binary tree
ii. Complete binary tree
iii. Extended binary tree



A. Strictly binary tree: A binary tree in which every node have exactly two or zero number of children is called strictly binary tree. It is also called as fully binary tree (or) proper BT (or) 2-tree.



Complete binary tree: In a complete binary tree, every internal node has exactly two children and all leaf nodes are at same leve

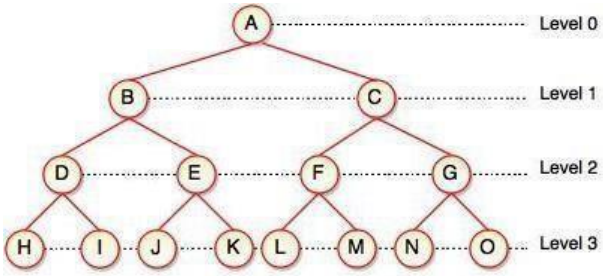
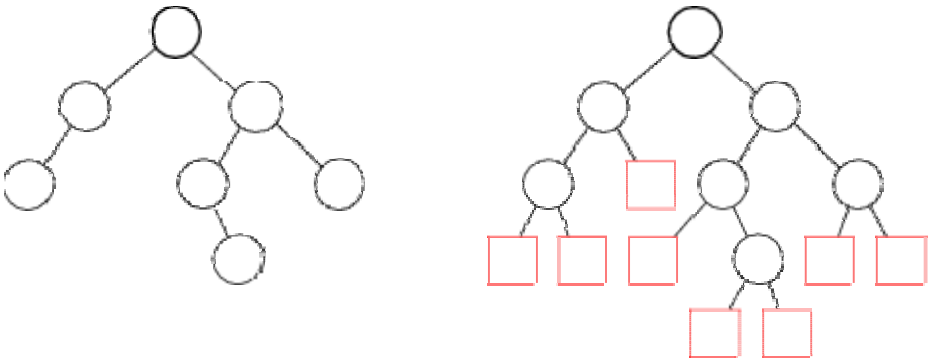


Fig. Complete Binary Tree

B. Extended binary tree: The fully Binary tree obtained by adding dummy node to a binary tree is called extended Binary tree.

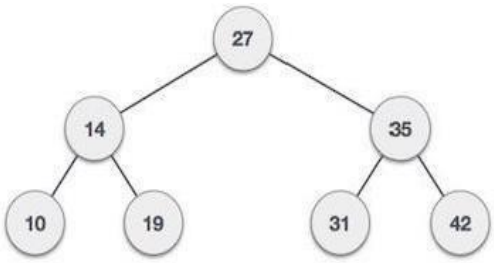


b. **Binary Search Tree (BST):**

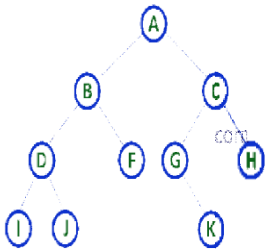
In Binary search tree, each node can have maximum of two children and it can be arranged as:

1. Any left chain node value less than its root value.
2. Any right node has a value greater than or equal to that of its root value.

Ex: 27,14,35,10,31,19,42



Representation of Binary trees using array and list



A binary tree data structure is represented using two methods. Those are:

1. Array Representation
2. Linked List Representation

Consider the following binary tree...

1. Array Representation of Binary Tree:

In array representation of a binary tree, we use one-dimensional array (1-D Array) to represent a binary tree.

The binary tree is represented in an array by storing each element at the array position corresponding to the number assigned to it.

To represent a binary tree of depth **'n'** using array representation, we need one dimensional array with a maximum size of **2n + 1**.

Consider the above example of a binary tree and it is represented as follows...

A	B	C	D	F	G	H	I	J	-	-	-	K	-	-	-	-	-	-	-	-

Linked List Representation of Binary Tree:

In a double linked list, every node consists of three fields.

First field for storing left child address.

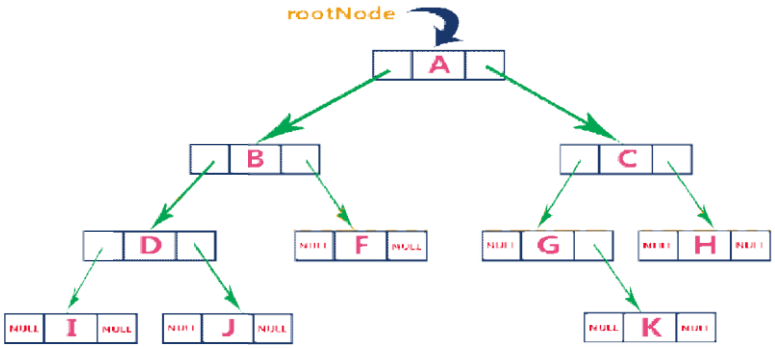
Second field for storing actual data.

Third field for storing right child address.

Left Child Address	Data	Right Child Address

In this linked list representation, a node has the following structure...

The above example of the binary tree represented using Linked list representation is shown as follows...



Binary tree

Tree: It is a non-linear DS, which organizes the data in hierarchical form. It contains connection of nodes which are connected a root node.

Binary tree: A binary tree (t) is a collection of nodes or elements. A binary tree is a tree where each node can have maximum of two children.

It is defined as:

It consists of a node called root node with two disjoint binary trees called left-subtree and right-subtree.

Binary tree :

The binary tree ADT performs the following Operations:

empty(): returns true if empty; returns false otherwise.

size(): returns number of elements/nodes in the tree.

pre-order(visit): pre-order traversal of binary tree.

in-order(visit): in-order traversal of binary tree.

post-order(visit): post-order traversal of binary tree.

level order(visit): level order traversal of binary tree.

Advantages of Binary Tree:

1.Searching in Binary tree become faster.
2.Binary tree provides six traversals.
3.Two of six traversals give sorted order of elements.
4. Maximum and minimum elements can be directly picked up.
5. It is used for graph traversal and to convert an expression to postfix and prefix forms.

Applications:

1. Implementing rooting tables in routers.
2. Data compression code.
3. Expression evaluation.
4. Implementation of expression parsers and expression solvers.

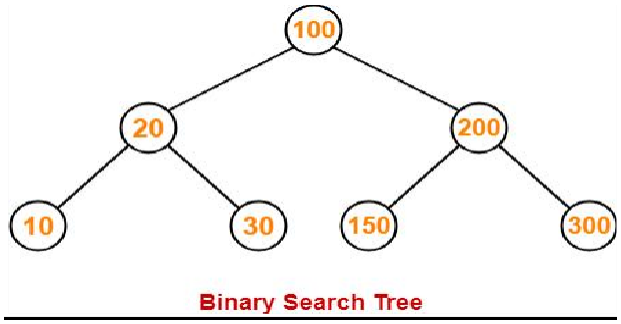
Explain about Binary Tree Traversals

Traversing is a process of visiting, searching or displaying elements in a tree. In a binary tree, displaying order of elements depending upon traversal methods.

Displaying or visiting order of nods in a binary tree is called as a binary tree traversal. There are 3 types of binary tree traversals. They are

- pre-order traversal
- In- order traversal
- Post-order traversal.

For Example: 100,200,20,150,10,30,300



Pre-order (root-left-right):

To traverse a binary tree in pre-order the following operations are performed:

1.Visit the root node.
2.Traverse the left sub-tree, i.e., call preorder(left-subtree)
3.Traverse the right sub-tree, i.e., call preorder(right-subtree)

Algorithm:

The algorithm using recursion of pre-order (node).

Step 1: visit node.

Step 2: if (node.left !=null) then pre-order (node.left).

Step 3: if (node.right !=null). The pre-order (node.right).

Preorder Traversal of above diagram is**:** 100 , 20 , 10 , 30 , 200 , 150 , 300

In-order (left-root-right):

To traverse the binary tree In-order following operations are performed.

1. Traverse the left sub-tree, Traverse the left sub-tree, i.e., call inorder(left-subtree)
2. Visit root-node
3. Traverse the right sub-tree, i.e., call inorder (right-subtree)

Algorithm:

The algorithm using recursion of In-order.

Step 1: if (node.left !=null) then In-order (node.left).

Step 2: visit (node).

Step 3: if (node.right != null) then In-order (node.right).

Inorder Traversal of above diagram is: 10 , 20 , 30 , 100 , 150 , 200 , 300

Post-order (left-right-root):

To traverse the binary tree in post-order following operations are performed.

1. Traverse the left sub-tree, i.e., call post-order(left-subtree)
2. Traverse the right sub-tree, i.e., call post-order(right-subtree)
3. Visit root-node.

Algorithm:

The algorithm using recursion of post-order.

Step 1: if (node.left != null) then post-order(node.left).

Step 2: if (node.right != null) then post-order(node.right).

Step 3: visit (node).

Postorder Traversal of above diagram is**:** 10 , 30 , 20 , 150 , 300 , 200 , 100

Java Program with All Traversals

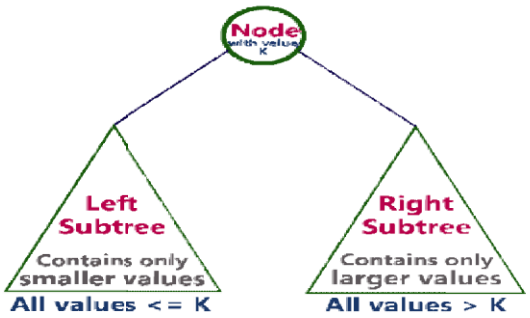
```
 import java.util.*; class Node {
 int data;
 Node left, right;
public Node(int item) {
 data = item;
 left = right = null;
   }  }
public class BinaryTreeTraversals {
 Node root;
void inorder(Node root) {
 if (root == null) return;
 inorder(root.left);
 System.out.print(root.data + " ");
 inorder(root.right);
}
void preorder(Node root) {
 if (root == null) return;
 System.out.print(root.data + " ");
 preorder(root.left);
 preorder(root.right);
}
void postorder(Node root) {
 if (root == null) return;
 postorder(root.left);
 postorder(root.right);
 System.out.print(root.data + " ");
}
}
```

```
public static void main(String[] args) {
    BinaryTreeTraversals tree = new BinaryTreeTraversals();
    // Constructing the tree
    tree.root = new Node(1);
    tree.root.left = new Node(2);
    tree.root.right = new Node(3);
    tree.root.left.left = new Node(4);
    tree.root.left.right = new Node(5);
    tree.root.right.right = new Node(6);
    System.out.println("Inorder Traversal:");
    tree.inorder(tree.root);
    System.out.println("\nPreorder Traversal:");
    tree.preorder(tree.root);
    System.out.println("\nPostorder Traversal:");
    tree.postorder(tree.root);
    System.out.println("\nLevel Order Traversal:");
    tree.levelOrder(tree.root);
}
 }
```

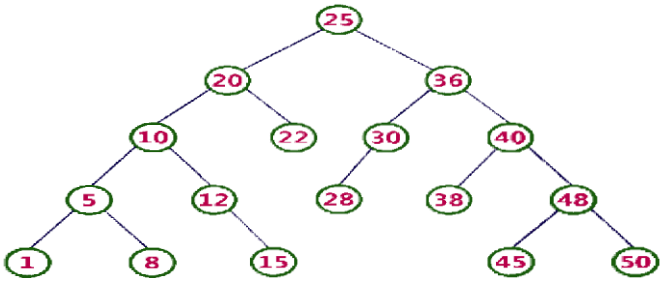
Inorder Traversal:
4 2 5 1 3 6
Preorder Traversal:
1 2 4 5 3 6
Postorder Traversal:
4 5 2 6 3 1

Binary search Trees

"Binary Search Tree is a binary tree in which every node contains only smaller values in its left subtree and only larger values in its right subtree."



Example:



Operations on a Binary Search Tree:

The following operations are performed on a binary search tree...

- Search
- Insertion
- Deletion

1. Search Operation in BST:

In a binary search tree, the search operation is performed with **O(log n)** time complexity. The search operation is performed as follows...

Step 1: Read the search element from the user.
Step 2: Compare the search element with the value of root node in the tree.
Step 3: If both are matched, then display "Given node is found!!!" and terminate the function
Step 4: If both are not matched, then check whether search element is smaller or larger than that node value.
Step 5: If search element is smaller, then continue the search process in left subtree.
Step 6: If search element is larger, then continue the search process in right subtree.
Step 7: Repeat the same until we find the exact element or until the search element is compared with the leaf node
Step 8: If we reach to the node having the value equal to the search value then display "Element is found" and terminate the function.
Step 9: If we reach to the leaf node and if it is also not matched with the search element, then display "Element is not found" and terminate the function.

2. Insertion Operation in BST:

In a binary search tree, the insertion operation is performed with **O(log n)** time complexity. In binary search tree, new node is always inserted as a leaf node.

The insertion operation is performed as follows...

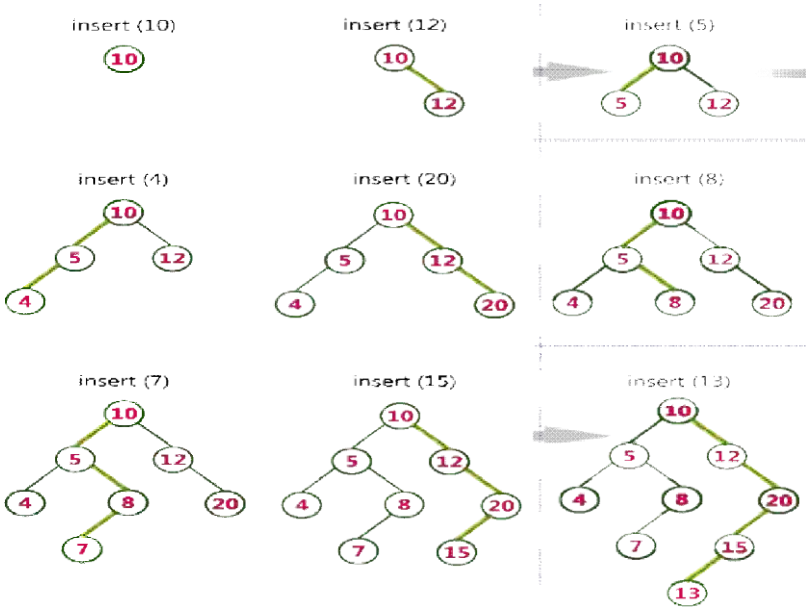
Step 1: Create a newNode with given value and set its **left** and **right** to **NULL**.
Step 2: Check whether tree is Empty.
Step 3: If the tree is **Empty**, then set **root** to **newNode**.
Step 4: If the tree is **Not Empty**, then check whether the value of newNode is **smaller** or **larger** than the node (here it is root node).
Step 5: If newNode is **smaller** than **or equal** to the node then move to its **left** child. If newNode is **larger** than the node then move to its **right** child.
Step 6: Repeat the above steps until we reach to the **leaf** node (i.e., reaches to NULL).
Step 7: After reaching the leaf node, insert the newNode as **left child** if the newNode is **smaller or equal** to that leaf node or else insert it as **right child**.

Example:

Construct a Binary Search Tree by inserting the following sequence of numbers...

10,12,5,4,20,8,7,15 and 13

Above elements are inserted into a Binary Search Tree as follows...



3. Deletion Operation in BST:

In a binary search tree, the deletion operation is performed with **O(log n)** time complexity. Deleting a node from Binary search tree includes following three cases...

Case 1: Deleting a Leaf node (A node with no children)

Case 2: Deleting a node with one child

Case 3: Deleting a node with two children

Case 1: Deleting a leaf node:

We use the following steps to delete a leaf node from BST...

Step 1: Find the node to be deleted using **search operation**

Step 2: Delete the node using **free** function (If it is a leaf) and terminate the function.

Case 2: Deleting a node with one child:

We use the following steps to delete a node with one child from BST...

Step 1: Find the node to be deleted using **search operation**

Step 2: If it has only one child then create a link between its parent node and child node.

Step 3: Delete the node using **free** function and terminate the function.

Case 3: Deleting a node with two children:

We use the following steps to delete a node with two children from BST...

Step 1: Find the node to be deleted using **search operation**

Step 2: The node which is to be deleted, is replaced with its in-order successor or predecessor node.

Step 3: Delete the node using **free** function and terminate the function.

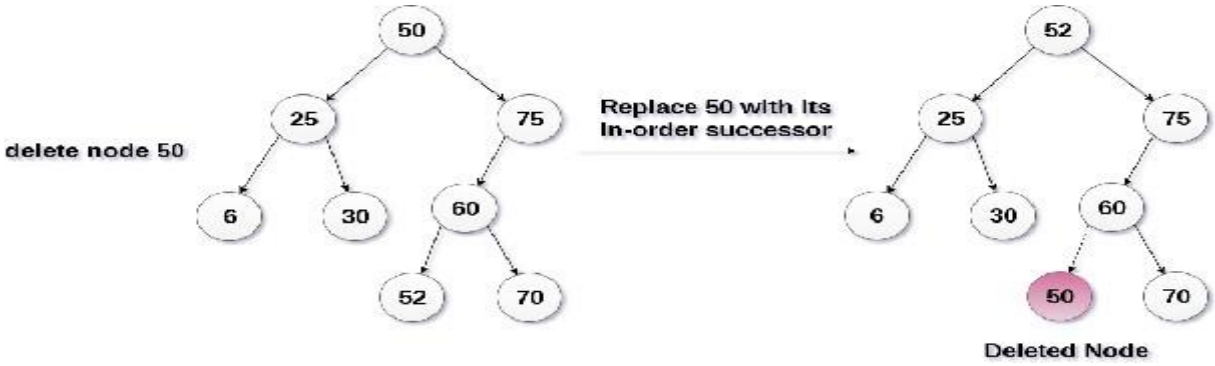
Example:

In the following image, the node 50 is to be deleted which is the root node of the tree.

The in-order traversal of the given tree is:

6, 25, 30, 50, 52, 60, 70, 75.

Replace 50 with its in-order successor 52. Now, 50 will be moved to the leaf of the tree, which will simply be deleted.



Applications and Advantages of BST

Applications of BST:

- Used to express arithmetic expressions
- Used to evaluate expression trees.
- Used for indexing IP addresses.
- Hashing would be faster, but want to avoid attacker sending IP packets with worst-case inputs.
- For dynamic sorting

Advantages:

1. BST is fast in insertion and deletion etc when balanced.
2. Very efficient and its code is easier than link lists.
3. Stores keys in the nodes in a way that searching, insertion and deletion can be done efficiently

AVL Tree

An **AVL tree** defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left And right subtrees for any node cannot be more than one.

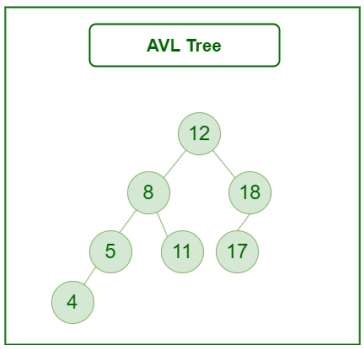
- The absolute difference between the heights of the left subtree and the right subtree for any node is
- known as the **balance factor** of the node. The balance factor for all nodes must be less than or equal to 1.
- Every AVL tree is also a Binary Search Tree.

Balance Factor (BF) Calculation

BF=Height of Left Subtree−Height of Right SubtreeBF = \text{Height of Left Subtree} - \text{Height of Right Subtree}BF=Height of Left Subtree−Height of Right Subtree

- **BF = 1, 0, or -1** → Tree is balanced ✅
- **BF < -1 or BF > 1** → Tree is unbalanced ❌ (Requires rotation)

The balance factors for different nodes are : 12 :1, 8:1, 18:1, 5:1, 11:0, 17:0 and 4:0. Since all differences are less than or equal to 1, the tree is an AVL tree.



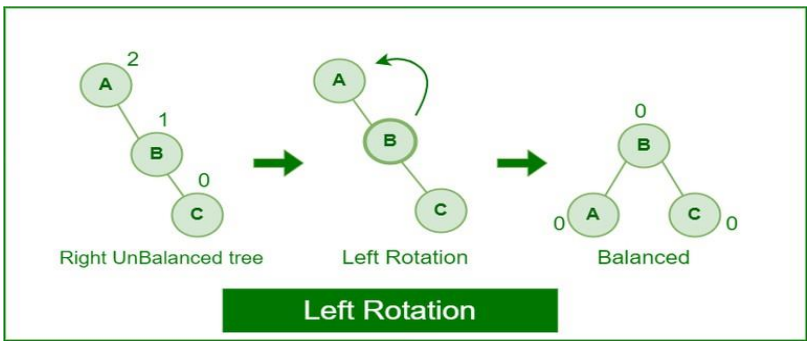
Rotations in AVL Tree

When an insertion causes imbalance, we use rotations:

1. **Right Rotation (Single Rotation - LL Case)**
2. **Left Rotation (Single Rotation - RR Case)**
3. **Left-Right Rotation (Double Rotation - LR Case)**
4. **Right-Left Rotation (Double Rotation - RL Case)**

Left Rotation:

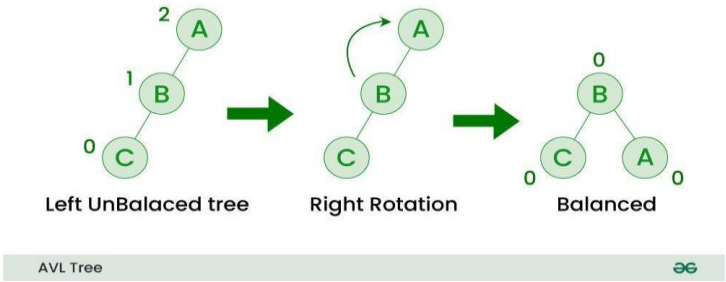
When a node is added into the right subtree of the right subtree, if the tree gets out of balance, we do a single left rotation.



Left-Rotation in AVL tree

Right Rotation:

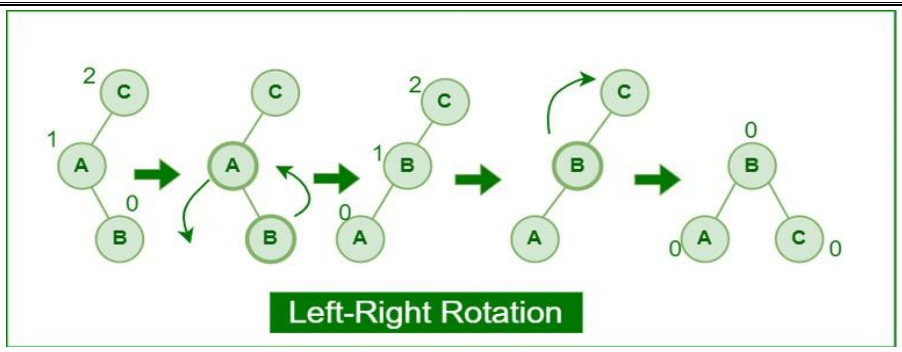
If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation.



Right-Rotation in AVL Tree

Left-Right Rotation:

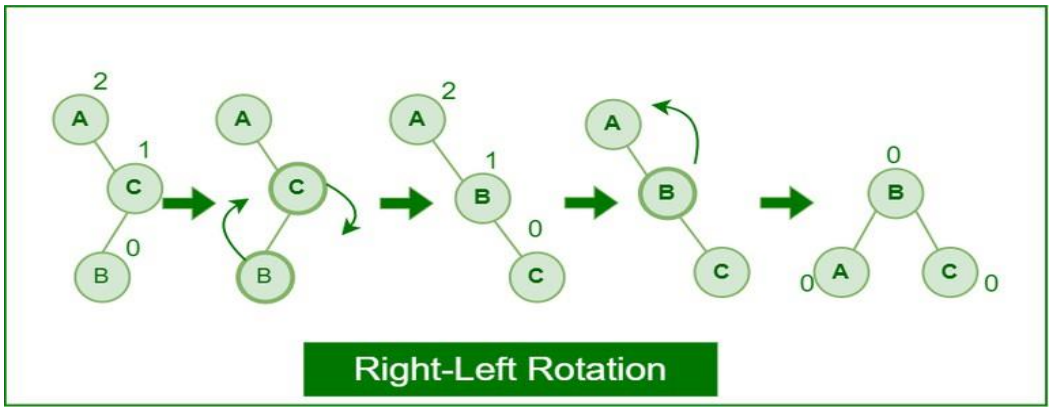
A left-right rotation is a combination in which first left rotation takes place after that right rotation executes.



Left-Right Rotation in AVL tree

Right-Left Rotation:

A right-left rotation is a combination in which first right rotation takes place after that left rotation executes.



Right-Left Rotation in AVL tree

Advantages of AVL Tree:

1. AVL trees can self-balance themselves and therefore provides time complexity as O(Log n) for search, insert and delete.
2. It is a BST only (with balancing), so items can be traversed in sorted order.
3. Since the balancing rules are strict compared to Red Black Tree, AVL trees in general have relatively less height and hence the search is faster.
4. AVL tree is relatively less complex to understand and implement compared to Red Black Trees.

Disadvantages of AVL Tree:

1. It is difficult to implement compared to normal BST and easier compared to Red Black
2. Less used compared to Red-Black trees. Due to its rather strict balance, AVL trees provide complicated insertion and removal operations as more rotations are performed.

Applications of AVL Tree:

1. AVL Tree is used as a first example self balancing BST in teaching DSA as it is easier to understand and implement compared to Red Black
2. Applications, where insertions and deletions are less common but frequent data lookups along with other operations of BST like sorted traversal, floor, ceil, min and max.
3. Red Black tree is more commonly implemented in language libraries like map in C++, set in C++, TreeMap in Java and TreeSet in Java.
4. AVL Trees can be used in a real time environment where predictable and consistent performance is required.

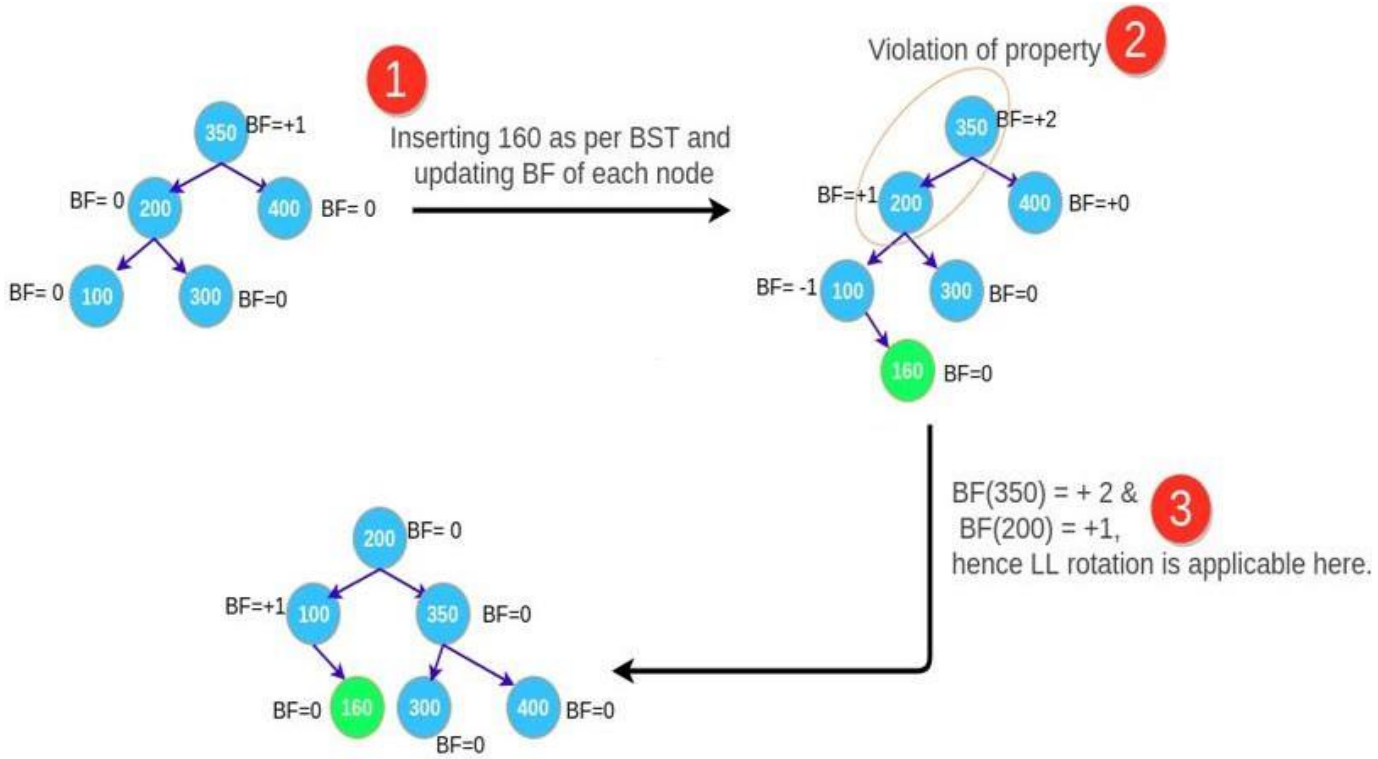
Insertion in AVL Trees

Insert operation is almost the same as in simple binary search trees. After every insertion, we balance the height of the tree. Insert operation takes O(log n) worst time complexity.

Step 1: Insert the node in the AVL tree using the same insertion algorithm of BST. In the above example, insert 160. **Step 2**: Once the node is added, the balance factor of each node is updated. After 160 is inserted, the balance factor of every node is updated.

Step 3: Now check if any node violates the range of the balance factor if the balance factor is violated, then perform rotations using the below case. In the above example, the balance factor of 350 is violated and case 1 becomes applicable there, we perform LL rotation and the tree is balanced again.

1. If BF(node) = +2 and BF(node -> left-child) = +1, perform LL rotation.
2. If BF(node) = -2 and BF(node -> right-child) = 1, perform RR rotation.
3. If BF(node) = -2 and BF(node -> right-child) = +1, perform RL rotation.
4. If BF(node) = +2 and BF(node -> left-child) = -1, perform LR rotation.



Deletion in AVL Trees

Deletion is also very straight forward. We delete using the same logic as in simple binary search trees. After deletion, we restructure the tree, if needed, to maintain its balanced height.

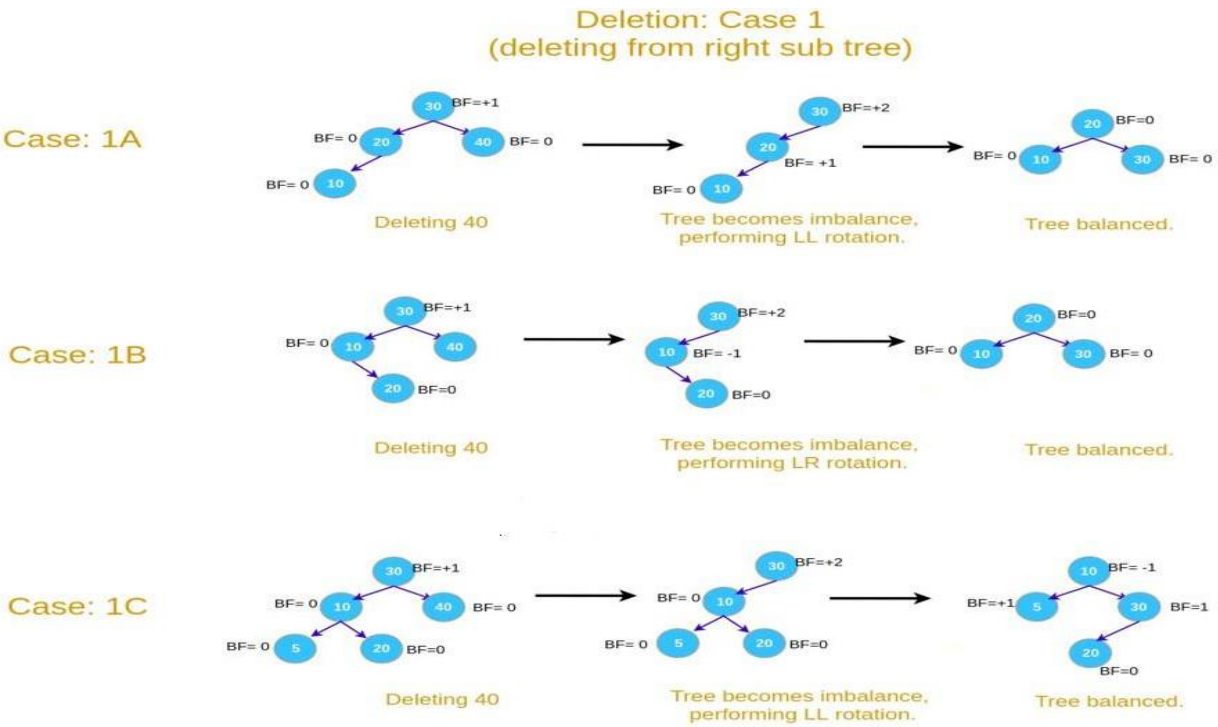
Step 1: Find the element in the tree.

Step 2: Delete the node, as per the BST Deletion.

Step 3: Two cases are possible:-

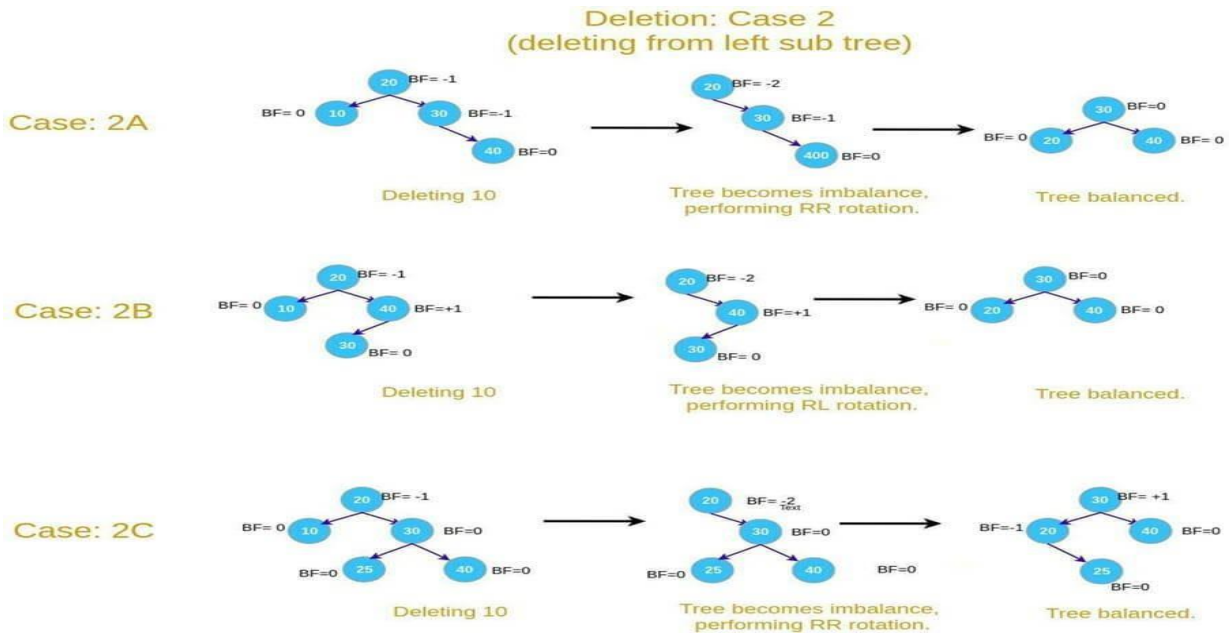
Case 1: Deleting from the right subtree.

- If BF(node) = +2 and BF(node -> left-child) = +1, perform LL rotation.
- If BF(node) = +2 and BF(node -> left-child) = -1, perform LR rotation.
- If BF(node) = +2 and BF(node -> left-child) = 0, perform LL rotation.



Case 2: Deleting from left subtree.

- 2A. If BF(node) = -2 and BF(node -> right-child) = -1, perform RR rotation.
- 2B. If BF(node) = -2 and BF(node -> right-child) = +1, perform RL rotation.
- 2C. If BF(node) = -2 and BF(node -> right-child) = 0, perform RR rotation.



Java Implementation of AVL Tree

```
class Node {
int data, height;
Node left, right;
public Node(int key) {
    data = key;
    height = 1;
}
}
class AVLTree
{
 Node root;
int height(Node N) {
    return (N == null) ? 0 : N.height;
}
int getBalance(Node N) {
    return (N == null) ? 0 : height(N.left) - height(N.right); }
Node rightRotate(Node y) {
    Node x = y.left;
    Node T2 = x.right;
    x.right = y;
    y.left = T2;
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    x.height = Math.max(height(x.left), height(x.right)) + 1;
```

```
    return x;
}
Node leftRotate(Node x) {
    Node y = x.right;
    Node T2 = y.left;
    y.left = x;
    x.right = T2;
    x.height = Math.max(height(x.left), height(x.right)) + 1;
    y.height = Math.max(height(y.left), height(y.right)) + 1;
    return y;
}
Node insert(Node node, int key) {
    if (node == null)
        return new Node(key);
    if (key < node.data)
        node.left = insert(node.left, key);
    else if (key > node.data)
        node.right = insert(node.right, key);
    else
        return node; // Duplicates not allowed
    node.height = 1 + Math.max(height(node.left), height(node.right));
    int balance = getBalance(node);
    if (balance > 1 && key < node.left.data)
        return rightRotate(node);
    if (balance < -1 && key > node.right.data)
        return leftRotate(node);
    if (balance > 1 && key > node.left.data) {
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node.right.data) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }
    return node;
}
void inorder(Node node) {
    if (node != null) {
        inorder(node.left);
        System.out.print(node.data + " ");
        inorder(node.right);
    }
}
```

```
public static void main(String[] args) {
    AVLTree tree = new AVLTree();
    tree.root = tree.insert(tree.root, 30);
    tree.root = tree.insert(tree.root, 20);
    tree.root = tree.insert(tree.root, 40);
    tree.root = tree.insert(tree.root, 10);
    tree.root = tree.insert(tree.root, 25);
    tree.root = tree.insert(tree.root, 35);
    tree.root = tree.insert(tree.root, 50);
    System.out.println("Inorder Traversal of AVL Tree:");
    tree.inorder(tree.root);
}
}
```

Example Execution

Inserted Nodes: 30, 20, 40, 10, 25, 35, 50

Balanced AVL Tree Structure

```
   30
  / \
 20   40
/ \ / \
10  25 35 50
```

Output:

Inorder Traversal of AVL Tree:

10 20 25 30 35 40 50

(The output is **sorted**, confirming the AVL tree property.)

Time Complexity

Operation	Time Complexity
Insertion	O(log N)
Deletion	O(log N)
Search	O(log N)

2-3 trees

Like other Trees include AVL trees, Red Black Tree, B tree, **2-3 Tree is also a height balanced tree**.

A **2-3 Tree** is a **self-balancing search tree** where:

- Each node has **one or two keys**.
- Nodes can have **two or three children**.
- The tree **remains balanced** at all times.

Rules of a 2-3 Tree

1. **Node Types**:
   - **2-Node**: Contains **one key** and has **two children**.
   - **3-Node**: Contains **two keys** and has **three children**.

2. **Sorting**: Keys inside a node are always stored in **sorted order**.
3. **Balanced Structure**:
   - Every **leaf node** is at the **same depth**.
   - No **skewed** branches.

Properties of 2-3 tree:

- Nodes with two children are called 2-nodes. The 2-nodes have one data value and two children
- Nodes with three children are called 3-nodes. The 3-nodes have two data values and three children.
- Data is stored in sorted order.
- It is a balanced tree.
- All the leaf nodes are at same level.
- Each node can either be leaf, 2 node, or 3 node.
- Always insertion is done at leaf.

Search: To search a key **K** in given 2-3 tree **T**, we follow the following procedure:

Base cases:

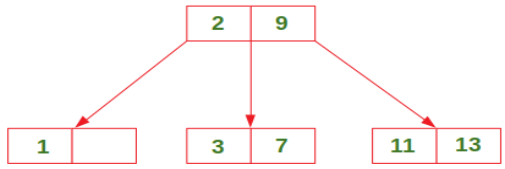
1. If **T** is empty, return False (key cannot be found in the tree).
2. If current node contains data value which is equal to **K**, return True.
3. If we reach the leaf-node and it doesn't contain the required key value **K**, return False.

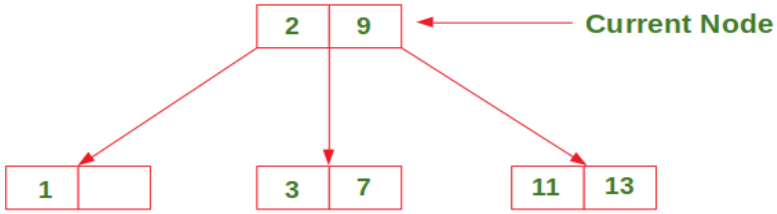
Recursive Calls:

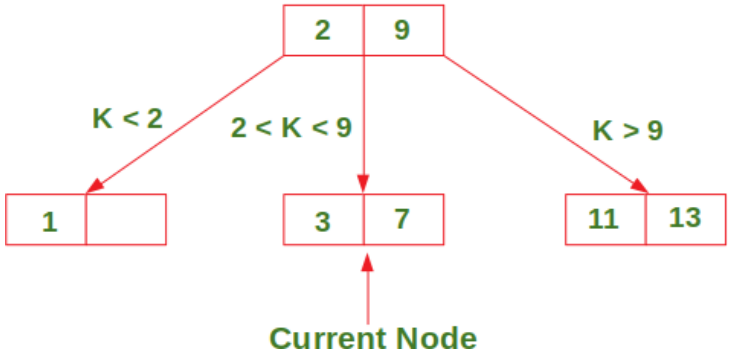
1. If **K** < currentNode.leftVal, we explore the left subtree of the current node.
2. Else if currentNode.leftVal < **K** < currentNode.rightVal, we explore the middle subtree of the current node.
3. Else if **K** > currentNode.rightVal, we explore the right subtree of the current node.

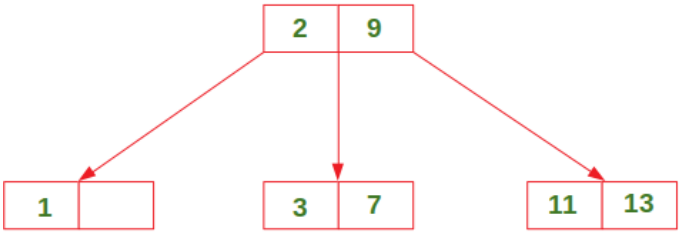
Consider the following example:

Search 5 in the following 2-3 Tree:







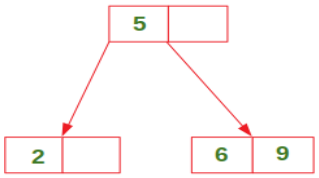


5 Not Found. Return False

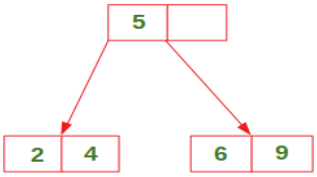
Insertion: There are 3 possible cases in insertion which have been discussed below:

- **Case 1:** Insert in a node with only one data element

Insert 4 in the following 2-3 Tree:



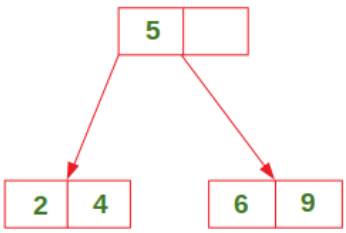
Initial



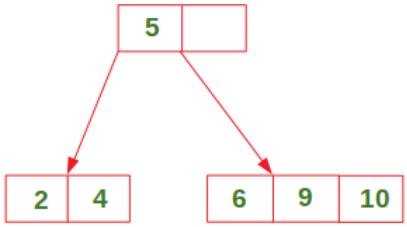
After Insertion

- **Case 2:** Insert in a node with two data elements whose parent contains only one data element.

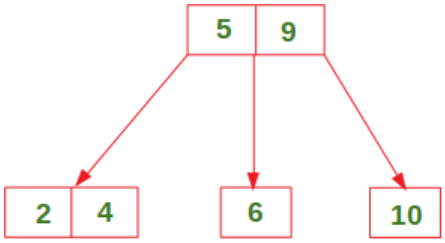
Insert 10 in the following 2-3 Tree:



Initial



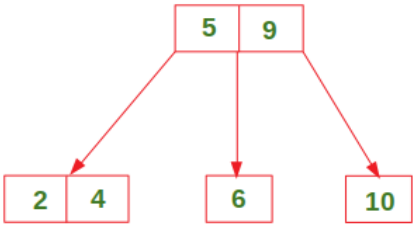
Temporary Node with 3 data elements



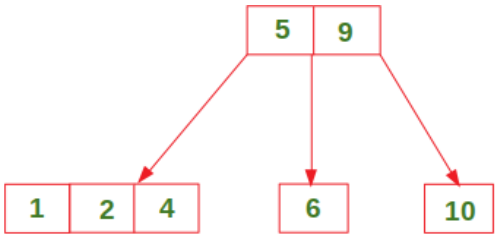
Move the middle element to
parent and split the current Node

- **Case 3:** Insert in a node with two data elements whose parent also contains two data elements.

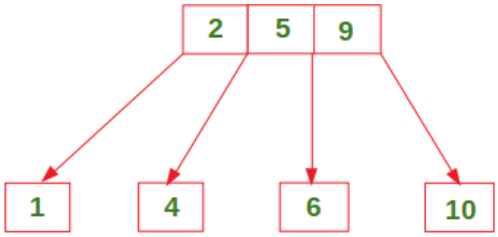
Insert 1 in the following 2-3 Tree:



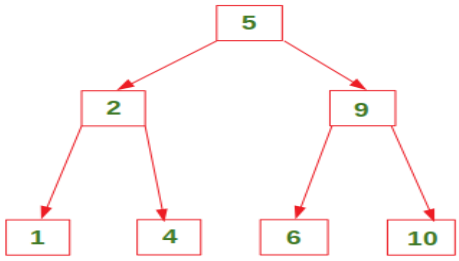
Initial



Temporary Node with 3 data elements



Move the middle element to the parent and split the current Node



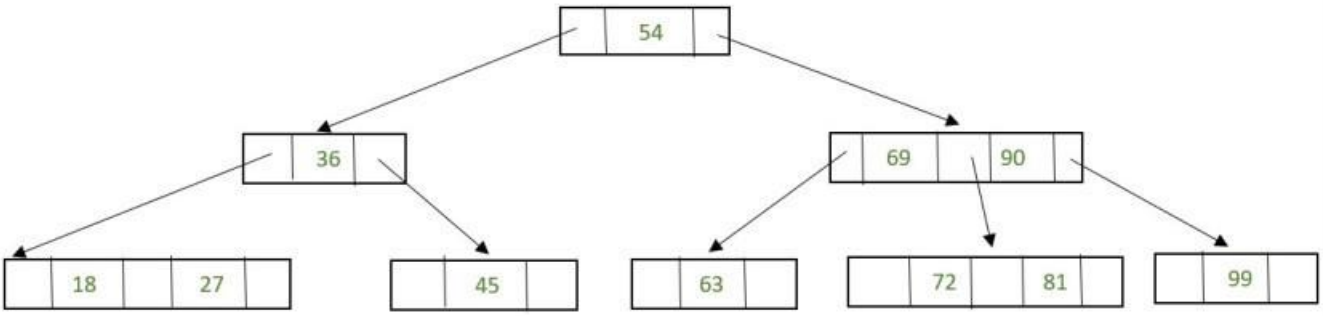
Move the middle element to the parent and split the current Node

In Deletion Process for a specific value:

- To delete a value, it is replaced by its in-order successor and then removed.
- If a node is left with less than one data value then two nodes must be merged together.
- If a node becomes empty after deleting a value, it is then merged with another node.

To Understand the deletion process-

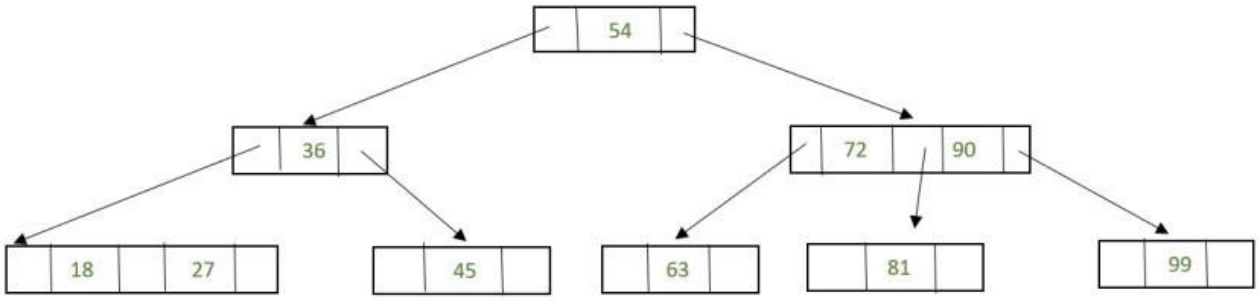
Consider the 2-3 tree given below



Given 2-3 Tree

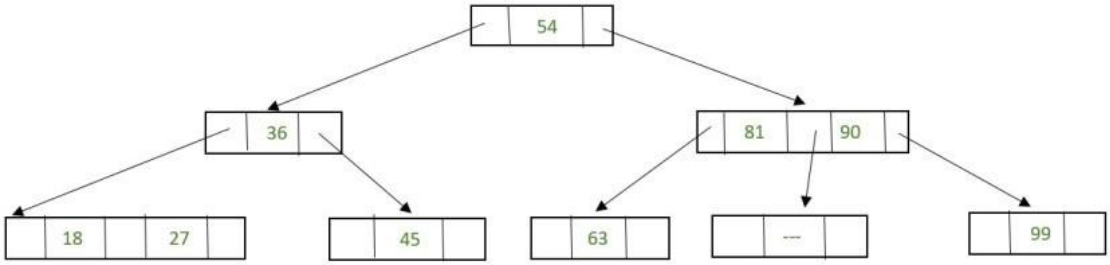
delete the following values from it: 69,72, 99, 81.

To delete 69, swap it with its in-order successor, that is, 72. 69 now comes in the leaf node. Remove the value 69 from the leaf node.



After deletion 69

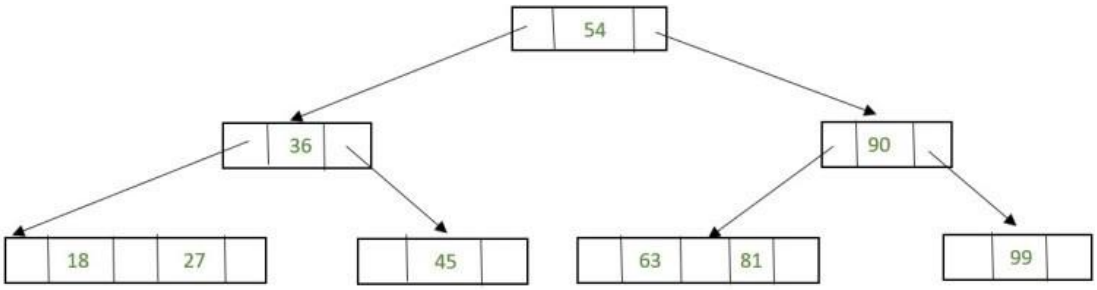
To delete 72, 72 is an internal node. To delete this value swap 72 with its in-order successor 81 so that 72 now becomes a leaf node. Remove the value 72 from the leaf node.



After deletion 72

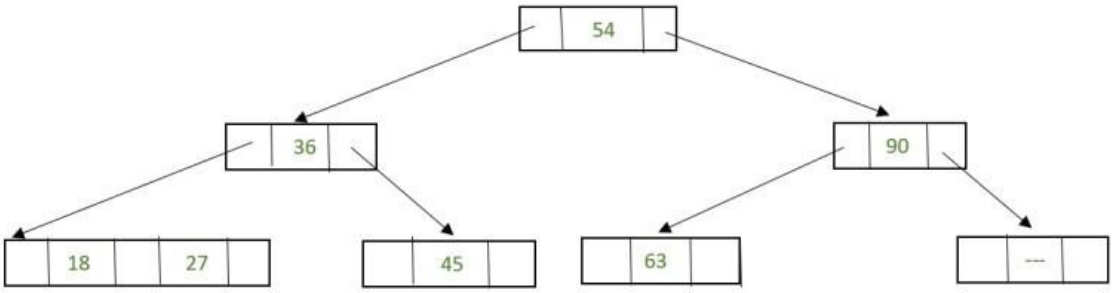
Now there is a leaf node that has less than 1 data value thereby violating the property of a 2-3 tree. So the node must be merged.

To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



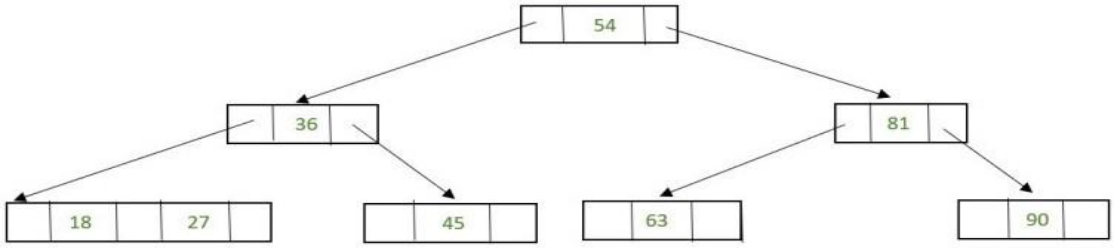
Rebalancing to Satisfy 23 Tree property

To delete 99, 99 is present in a leaf node, so the data value can be easily removed.



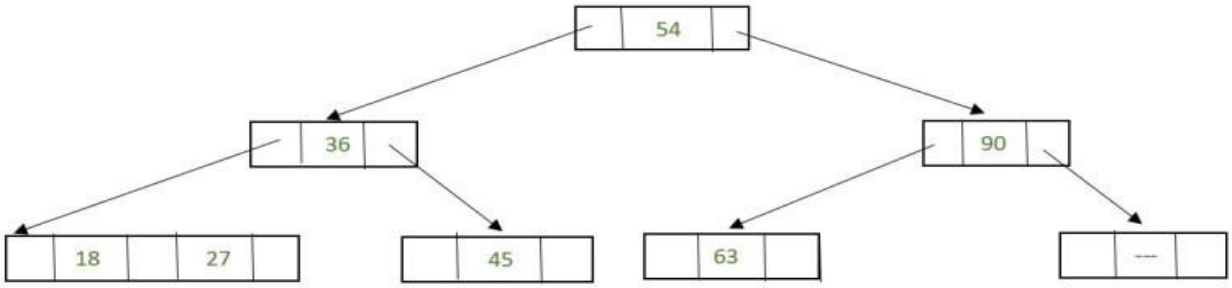
After deletion 99

Now there is a leaf node that has less than 1 data value, thereby violating the property of a 2-3 tree.
So the node must be unmerged. To merge the node, pull down the highest data value in the parent's node and unmerge it with its left sibling.



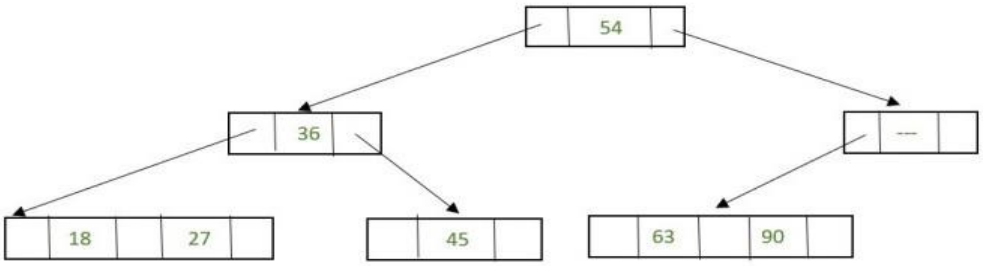
Rebalancing to Satisfy 2-3 Tree Property

To delete 81, 81 is an internal node. To delete this value swap 81 with its in-order successor 90 so that 81 now becomes a leaf node. Remove the value 81 from the leaf node.



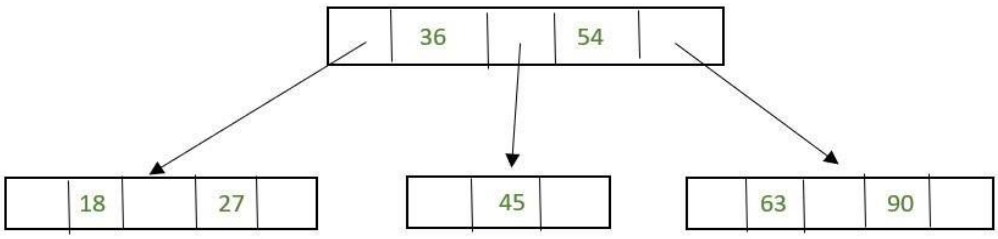
After deletion 81

Now there is a leaf node that has less than 1 data value, thereby violating the property of a 2-3 tree. So the node must be merged. To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



Rebalancing to Satisfy 2-3 Tree property

As internal node cannot be empty. So now pull down the lowest data value from the parent's node and merge the empty node with its left sibling



Rebalancing to Satisfy 2-3 Tree Property

NOTE: In a 2-3 tree, each interior node has either two or three children. This means that a **2-3 tree is not a binary tree.**

Heaps Tree

- Heap data structure is a specialized binary tree-based data structure.
- In a heap data structure, nodes are arranged based on their values.
- A heap data structure sometimes also called as Binary Heap.
- Every heap data structure has the following properties...

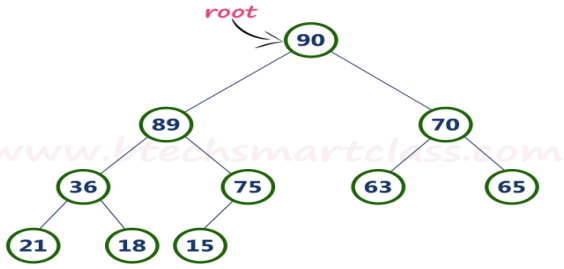
  Property 1 (Ordering): Nodes must be arranged in an order according to their values based on Max heap or Min heap.

  Property 2 (Structural): All levels in a heap must be full except the last level and all nodes must be filled from left to right strictly.
- There are two types of heap data structures and they are as follows...
  - **Max Heap**
  - **Min Heap**

Max Heap:

Max heap is a specialized full binary tree in which every parent node contains greater or equal value than its child nodes.

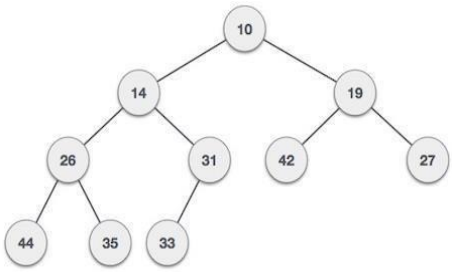
Example



Above tree is satisfying both Ordering property and Structural property according to the Max Heap data structure.

Min Heap:

Min heap is a specialized full binary tree in which every parent node contains lesser or equal value than its child nodes.



Operations on Heap Tree

The following operations are performed on a Max heap data structure...

Finding Maximum Value Operation in Max Heap:

Finding the node which has maximum value in a max heap is very simple. In a max heap, the root node has the maximum value than all other nodes. So, directly we can display root node value as the maximum value in max heap.

Insertion Operation in Max Heap:

Insertion Operation in max heap is performed as follows...

Step 1: Insert the **newNode** as **last leaf** from left to right.

Step 2: Compare **newNode value** with its **Parent node**.

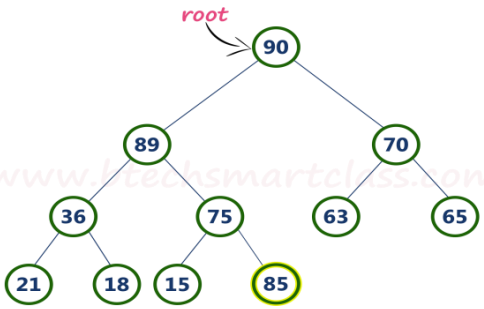
Step 3: If **newNode value is greater** than its parent, then **swap** both of them.

Step 4: Repeat step 2 and step 3 until newNode value is less than its parent node (or) newNode reaches to root.

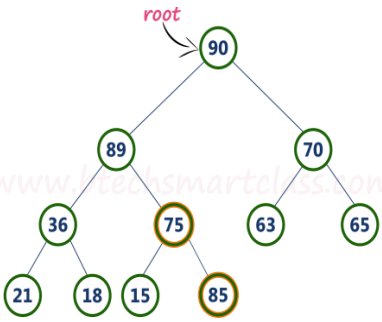
Example

Consider the above max heap. **Insert a new node with value 85.**

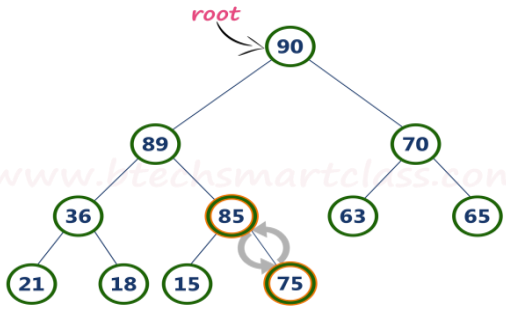
Step 1: Insert the **newNode** with value 85 as **last leaf** from left to right. That means newNode is added as a right child of node with value 75. After adding max heap is as follows...



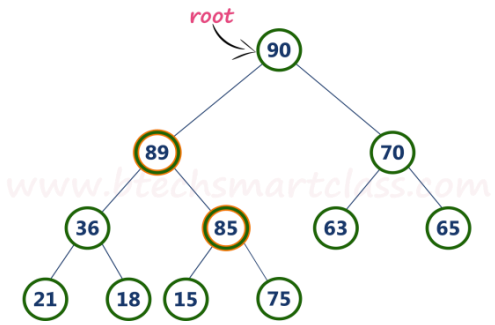
Step 2: Compare **newNode value (85)** with its **Parent node value (75)**. That means **85 > 75**



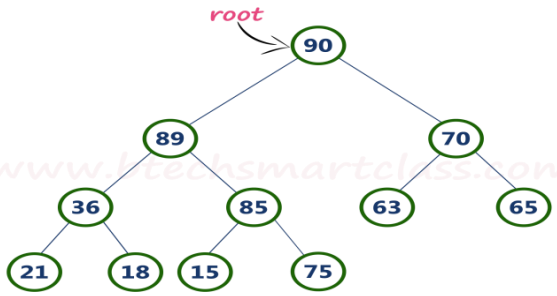
Step 3: Here **newNode value (85) is greater** than its **parent value (75)**, then **swap** both of them. After swapping, max heap is as follows...



Step 4: Now, again compare newNode value (85) with its parent node value (89).



Here, newNode value (85) is smaller than its parent node value (89). So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



Deletion Operation in Max Heap:

In a max heap, deleting the last node is very simple as it does not disturb max heap properties.

Deleting root node from a max heap is little difficult as it disturbs the max heap properties. We use the following steps to delete the root node from a max heap...

Step 1: **Swap** the **root** node with **last** node in max heap Step 2: **Delete** last node.

Step 3: Now, compare **root value** with its **left child value**.

Step 4: If **root value is smaller** than its left child, then compare **left child** with its **right sibling**. Else goto **Step 6**

Step 5: If **left child value is larger** than its **right sibling**, then **swap root** with **left child** otherwise **swap root** with its **right child**.

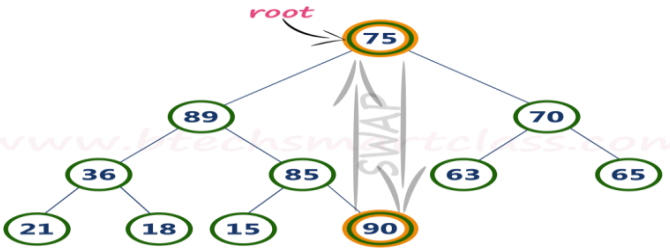
Step 6: If **root value is larger** than its left child, then compare **root value** with its **right child** value.

Step 7: If **root value is smaller** than its **right child**, then **swap root** with **right child** otherwise **stop the process**.

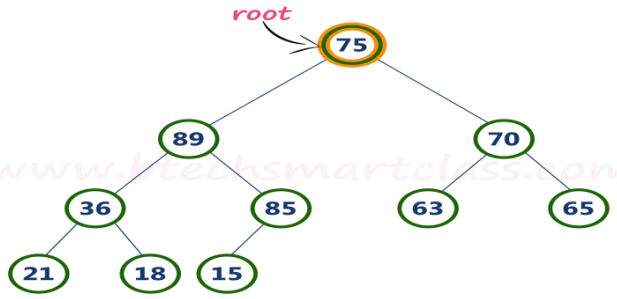
Step 8: Repeat the same until root node fixes at its exact position.

Example:Consider the above max heap. Delete root node (90) from the max heap.

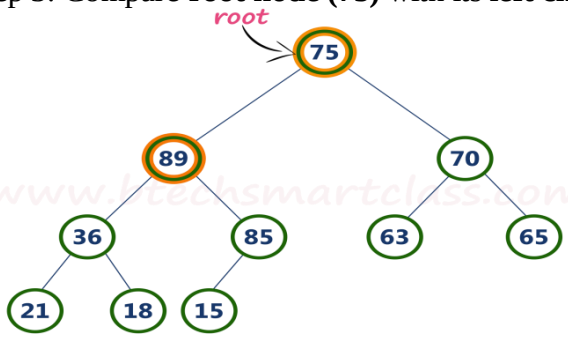
Step 1: **Swap** the **root node (90)** with **last node 75** in max heap. After swapping max heap is as follows...



Step 2: **Delete** last node. Here the last node is 90. After deleting node with value 90 from heap, max heap is as follows...

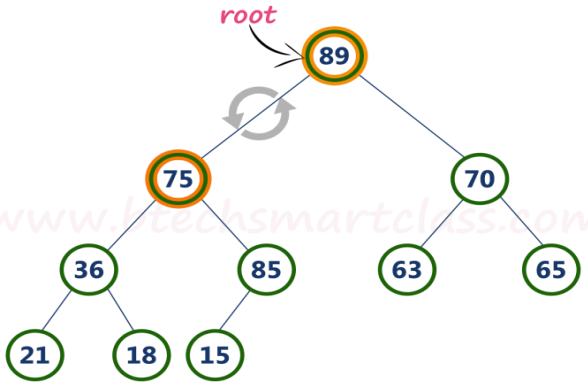


Step 3: Compare **root node (75)** with its **left child (89)**.

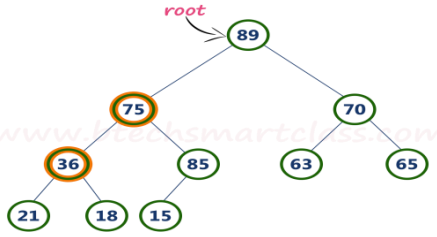


Here, **root value (75) is smaller** than its left child value (89). So, compare left child (89) with its right sibling (70).

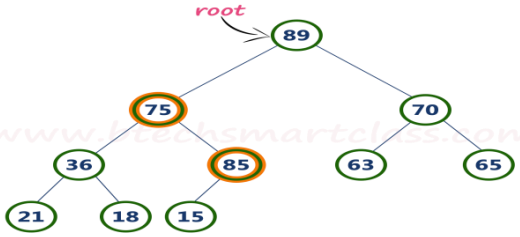
Step 4: Here, **left child value (89) is larger** than its **right sibling (70)**, So, **swap root (75)** with **left child (89)**.



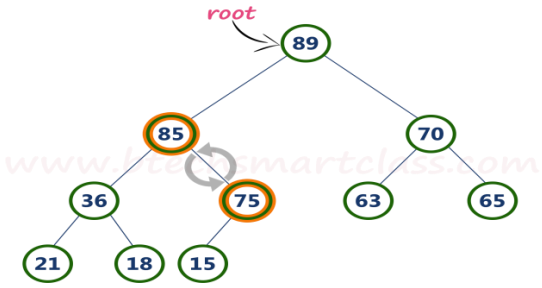
Step 5: Now, again compare **75** with its **left child (36)**.



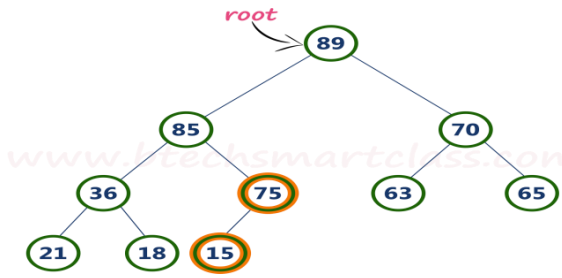
Here, node with value **75** is larger than its left child. So, we compare node **75** with its right child **85**.



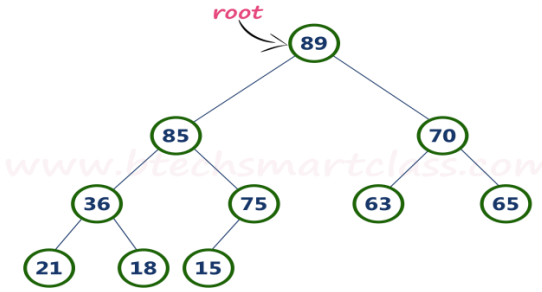
Step 6: Here, node with value **75** is smaller than its **right child (85)**. So, we swap both of them. After swapping max heap is as follows...



Step 7: Now, compare node with value **75** with its left child (**15**).



Here, node with value **75** is larger than its left child (**15**) and it does not have right child. So we stop the process. Finally, max heap after deleting root node (**90**) is as follows...



Note::Operations of min heap is same like as max heap.